Accumulating 128 out of 512 Output Channels in parallel

```
%4 = scf.for %arg1 = %c0 to %c512 step %c128 iter_args(%arg2 = %3) -> (tensor<1x7x7x512xf32>) {
  %weights = tensor.extract_slice %cst_0[0, 0, 0, %arg1] [3, 3, 512, 128] [1, 1, 1, 1] : tensor<3x3x512x512xf32> to tensor<3x3x512x128xf32>
  %output = tensor.extract_slice %arg2[0, 0, 0, %arg1] [1, 7, 7, 128] [1, 1, 1, 1] : tensor<1x7x7x512xf32> to tensor<1x7x7x128xf32>
  %7 = scf.for %arg3 = %c0 to %c128 step %c128 iter_args(%arg4 = %output) -> (tensor<1x7x7x128xf32>) {
    %8 = scf.for %arg5 = %c0 to %c7 step %c1 iter_args(%arg6 = %arg4) -> (tensor<1x7x7x128xf32>) {
      %10 = scf.for %arg7 = %c0 to %c6 step %c2 iter_args(%arg8 = %arg6) -> (tensor<1x7x7x128xf32>) {
        %11:2 = scf.for %arg9 = %c0 to %c3 step %c1 iter_args(%arg10 = %cst, %arg11 = %cst) -> (vector<128xf32>, vector<128xf32>) {
          %15:2 = scf.for %arg12 = %c0 to %c3 step %c1 iter_args(%arg13 = %arg10, %arg14 = %arg11) -> (vector<128xf32>, vector<128xf32>) {
            %16:2 = scf.for %arg15 = %c0 to %c512 step %c1 iter_args(%arg16 = %arg13, %arg17 = %arg14) -> (vector<128xf32>, vector<128xf32>) {
              %weightVector = vector.transfer_read %weights[%arg9, %arg12, %arg15, %arg3], %cst_1 {in_bounds = [true]}
              : tensor<3x3x512x128xf32>, vector<128xf32>
              %18 = arith.addi %arg5, %arg9 : index
              %19 = arith.addi %arg7, %arg12 : index
              %inputScalar0 = tensor.extract %padded[%c0, %18, %19, %arg15] : tensor<1x9x9x512xf32>
              %inputVector0 = vector.broadcast %inputScalar0 : f32 to vector<128xf32>
              %21 = vector.fma %weightVector, %inputVector0, %arg16 : vector<128xf32>
              %22 = arith.addi %arg7, %c1 : index
              %23 = arith.addi %arg5, %arg9 : index
              %24 = arith.addi %22, %arg12 : index
              %inputScalar1 = tensor.extract %padded[%c0, %23, %24, %arg15] : tensor<1x9x9x512xf32>
              %inputVector1 = vector.broadcast %inputScalar1 : f32 to vector<128xf32>
              %26 = vector.fma %weightVector, %inputVector1, %arg17 : vector<128xf32>
            }
          }
        }
        ...
          // Store the two finished accumulators
        ...
      }
        ...
        // Remainder handling with full scalar loop nest, since 7 % 2 = 1
        ...
    }
  }
```

Single Weight Vector reused across the Spatial Tile (size 2)